

ICLR 2021 | Paper

RODE: Learning **RO**les to DEcompose Multi-Agent Tasks

Tonghan Wang, Tarun Gupta, Anuj Mahajan, Bei Peng, Shimon Whiteson, et al.

PRESENTED BY

Inoussa Adouba

| Agenda

1. Motivation and Problem

2. Background Concepts

3. RODE Architecture

4. Results and Takeaways

General Information

Paper Title

RODE: Learning Roles to Decompose Multi-Agent Tasks

Authors & Affiliations

Tonghan Wang, Tarun Gupta, et al.
University of Oxford | Tsinghua University

Venue

ICLR 2021



Source: [1]

| The Core Question

How can many agents cooperate without searching the full joint action space?

Combinatorial Explosion

Exponentially growing joint action space

Exploration Crisis

Effective coordination becomes extremely difficult to discover

RODE's Solution

Decompose tasks into role-specific action spaces

| Real-World MARL

While StarCraft II is a standard benchmark, **Scalable Cooperation** is a critical challenge for real-world autonomy.

- Decentralized Sensor Networks
- Aerial Vehicles
- Collaborative Warehouse Robotics



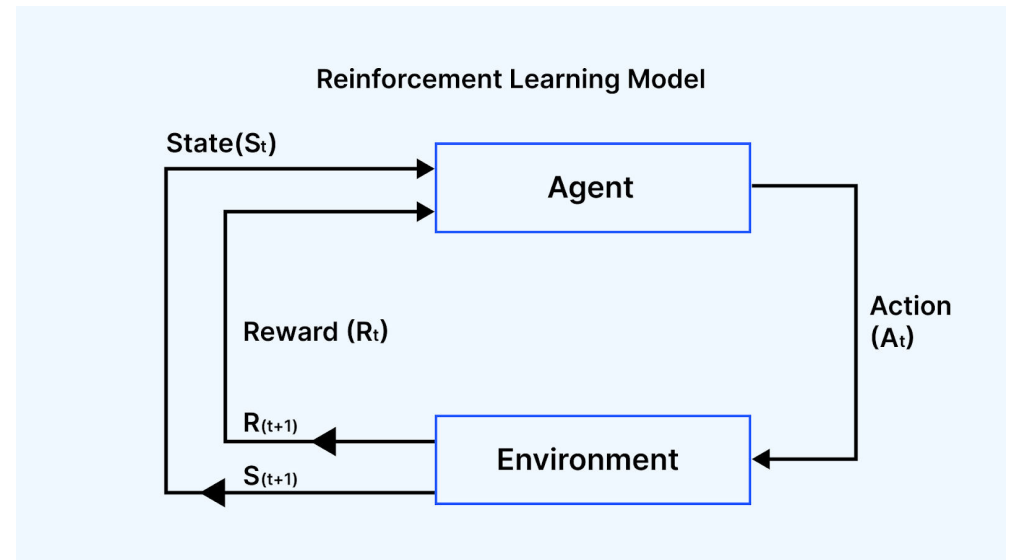
Source: [2]

| Recap: Reinforcement Learning

State: A description of the environment from the agent's perspective.

Action: The choice an agent makes to interact with the environment.

Reward: Scalar feedback signals indicate how well the agent is performing



Source: [3]

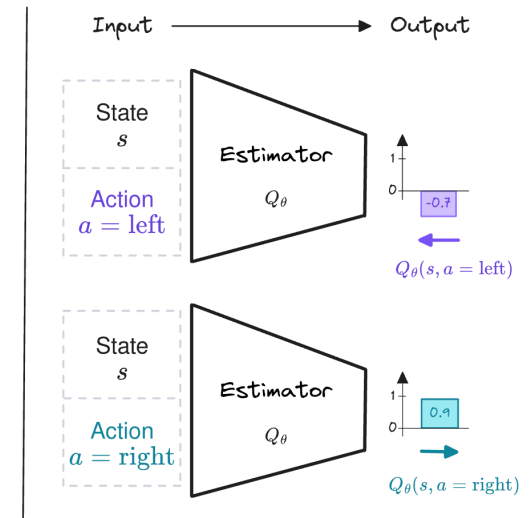
| From Q-Tables to Deep RL

Tabular RL requires an explicit entry for every possible state-action pair

Deep RL replaces Tabular RL with a neural network function approximator $Q(s, a; \theta)$ to predict values based on input features.

network weights update by minimizing Temporal Difference (TD) error instead of overwriting table cells.

Action \ State	$a = \text{left}$ ←	$a = \text{right}$ →
$s = 1$	$Q(s = 1, a = \text{left})$ 0.8	$Q(s = 1, a = \text{right})$ 1.2



Source: [4]

| From Deep RL to MARL

Deep RL Success

Function Approximation

Neural networks estimate Q-values instead of storing them in a table

Robust Generalization

Estimates Q-values even for unseen situations

The MARL Bottleneck

Non-Stationary

Other learning agents make the environment constantly change

Coordination Crisis

The real challenge is choosing actions together as combinations explode

| Joint Action Explosion

$$|A|^n$$

Total Combinatorial Search Space

$|A|$ = actions per agent, n = number of agents

10 actions and 8 agents = $10^8 = 100.000.000$ Joint actions

| Single-Agent - MDPs

Markov Decision Process (MDP)

A **Markov Decision Process** is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$:

- $\mathcal{S}$ is a (finite) set of states ($s \in \mathcal{S}$)
- $\mathcal{A}$ is a (finite) set of actions ($a \in \mathcal{A}$)
- $\mathcal{P}^a$ is a transition model that specifies $p(s'|s, a) = P(S_{t+1} = s' | S_t = s, A_t = a)$
 - in our case $\mathcal{P}^a$ is a state transition probability matrix
- $\mathcal{R}^a$ is a reward function, $\mathcal{R}_s^a = E[R_{t+1} | S_t = s, A_t = a]$
- γ is a discount factor, $\gamma \in [0, 1]$

Cooperative MARL Setting

Multi-Agent System



A team of N decentralized agents

Interacting with a shared environment

Shared Reward



cooperative task where all agents receive the same global reward signal

R

Partial Observations



Agents observe local views O_i instead of the full global state S

FRAMEWORK: Decentralized Partially Observable Markov Decision Process (Dec-POMDP)

Dec-POMDP Setup

$$G = \langle I, S, A, P, R, \Omega, O, n, \gamma \rangle$$

I, n, γ

Finite set I of n agents, and discount factor $\gamma \in [0, 1)$.

$S, s \in S$

The true, global **state** of the environment

$A, a \in A^n, P(s'|s,a)$

Each agent selects action $a_i \in A$, forming the **joint action** a . The environment transitions to state s' via probability P

$\Omega, O(s, i)$

Ω : observation space
 $O(s,i)$: local observation of agent i in state s

$R(s, a)$

The scalar **shared reward** r distributed to all agents

$\tau_i \in T \equiv (\Omega \times A)^*$

τ_i : local action-observation history

Because agents do not observe the full state s , their policies condition on τ_i instead.

Centralized Training, Decentralized Execution

Training

- Global State s
- All Agent Observations
- Shared Reward
- QMIX

Execution

- Agent 1 $\rightarrow \tau_1$
- Agent 2 $\rightarrow \tau_2$
- Agent $n \rightarrow \tau_n$

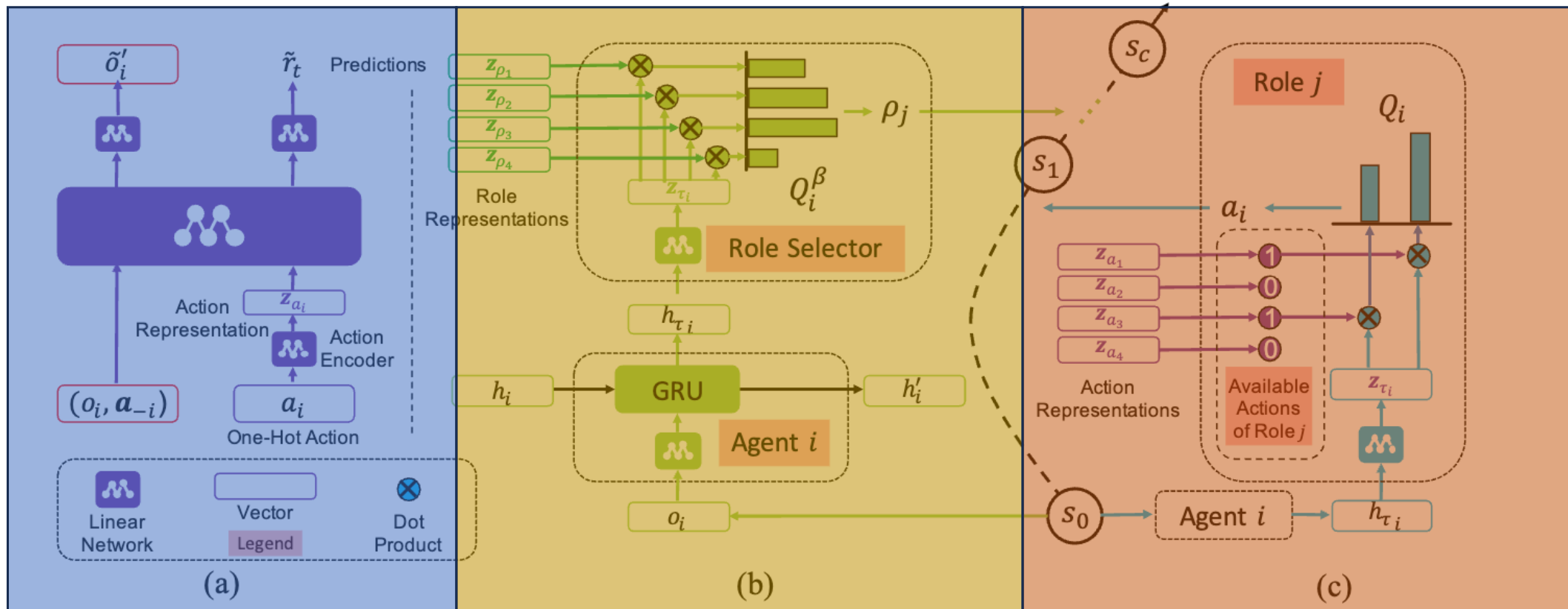
TRAIN TOGETHER $\rightarrow$ ACT ALONE

RODE's Baseline

Method	Key Idea	Limitation
VDN	Sum individual Q-values	Weak coordination
QMIX	Monotonic mixing network	Full primitive action space remains
ROMA	Emergent role learning	No explicit action-space reduction
RODE	Action-effect based roles	

RODE keeps CTDE but changes how the action space is explored.

RODE Framework



Source: [6]

Block 1

Block 2

Block 3

Block 1

How does RODE discover meaningful roles?

B1: Action Representation Forward Model

Goal

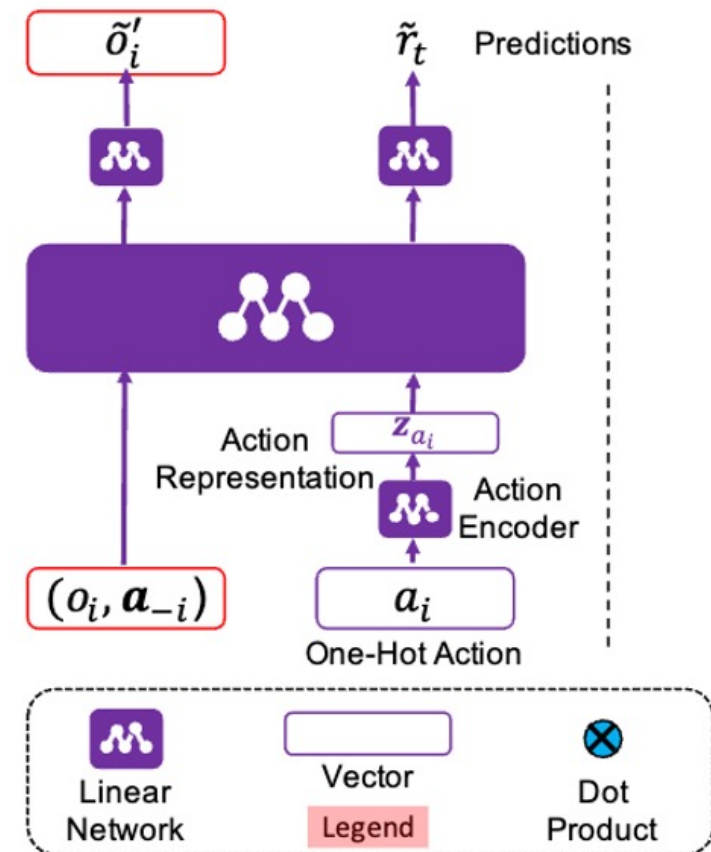
Learn action-effect embeddings

Inputs

- o_i, a_{-i}, a_i

Predictions

- Next observation $\tilde{o}'_i$ + Reward $\tilde{r}_t$

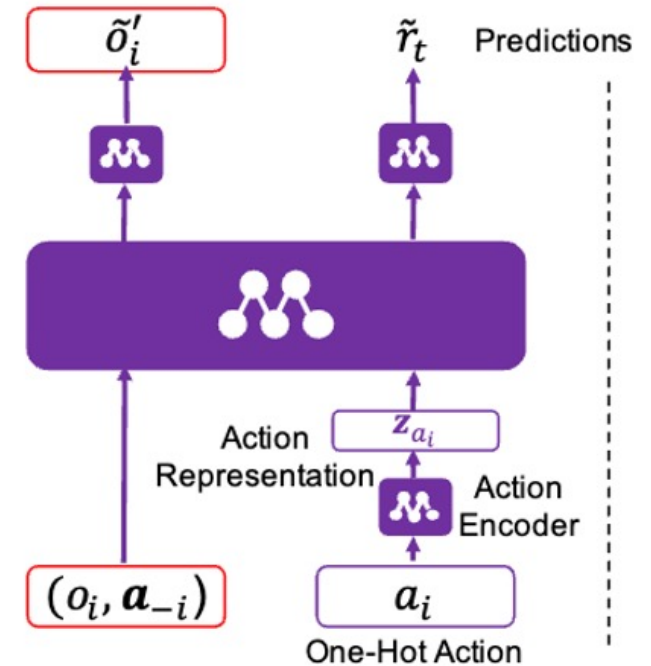


Source: [6]

B1: Training The Forward Model

Training Objective

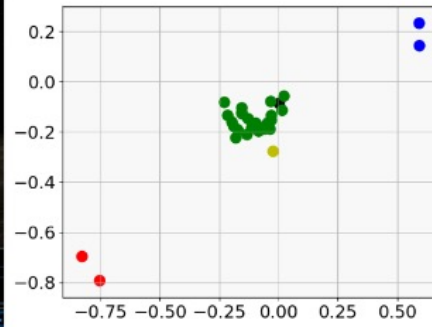
- Predict what happens next
- Compare prediction with target
- Minimize MSE error
- Backprop updates the action encoder



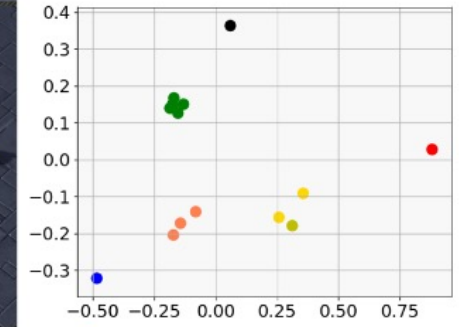
Source: [6]

$$\mathcal{L}_e(\theta_e, \xi_e) = \mathbb{E}_{(o, \mathbf{a}, r, o') \sim \mathcal{D}} \left[\sum_i \|p_o(\mathbf{z}_{a_i}, o_i, \mathbf{a}_{-i}) - o'_i\|_2^2 - \lambda_e \sum_i (p_r(\mathbf{z}_{a_i}, o_i, \mathbf{a}_{-i}) - r)^2 \right]$$

B1: Learned Action Representations



● ● Move in four directions ● Stop ● Attack different units ● Noop



● ● ● Move ● Stop ● Attack Zealots ● Attack Stalkers ● Noop

Source: [7]

| B1: From Embeddings to Roles

$$z_a \rightarrow k - \text{means} \rightarrow A_j$$

Step 1: Learn action embeddings

Step 2: Cluster actions by effect

Step 3: Each cluster becomes a role action space A_j

| B1: Role Action Spaces

Role 1 (Zealot Combat)

- Attack Zealot 1
- Attack Zealot 2
- Attack Zealot 3

Role 2 (Stalker Combat)

- Attack Stalker 1
- Attack Stalker 2

Example Clusters

| RODE's Role Definition

$$\rho_j = \langle g_j, \pi_{\rho_j} \rangle$$

The Sub-Task g_j

$$g_j = \langle I_j, S, A_j, \textcolor{red}{P}, R, \Omega_j, O, \gamma \rangle$$

- Inherits the global environment $(S, \textcolor{red}{P}, R, \gamma)$
- **Agent Subset:** $I_j \subset I$ (Agents currently assigned to role j)
- **Restricted Action Space:** $A_j \subset A$ (The key to scaling).

The Role Policy (π_{ρ_j})

$$\pi_{\rho_j} : T \times A_j \rightarrow [0, 1]$$

- Learns to solve the specific sub-task g_j .
- Acts **only** within the restricted action space A_j .

| B1: Frozen Role Action Spaces

Collect 50K timesteps



learn action embeddings z_a



k-means -> role action spaces A_j



Freeze embeddings + clusters

| B1: Frozen Role Action Spaces



Important

- Total training: $T \approx 2,000,000$ timesteps.
- Freezing occurs at just $\approx 2.5\%$ of total training time.
- Clusters are **never** unfrozen.

Block 2

Bi-Level Hierarchy

B2: Role Selector

Goal

Select the most useful role for the current situation

Inputs

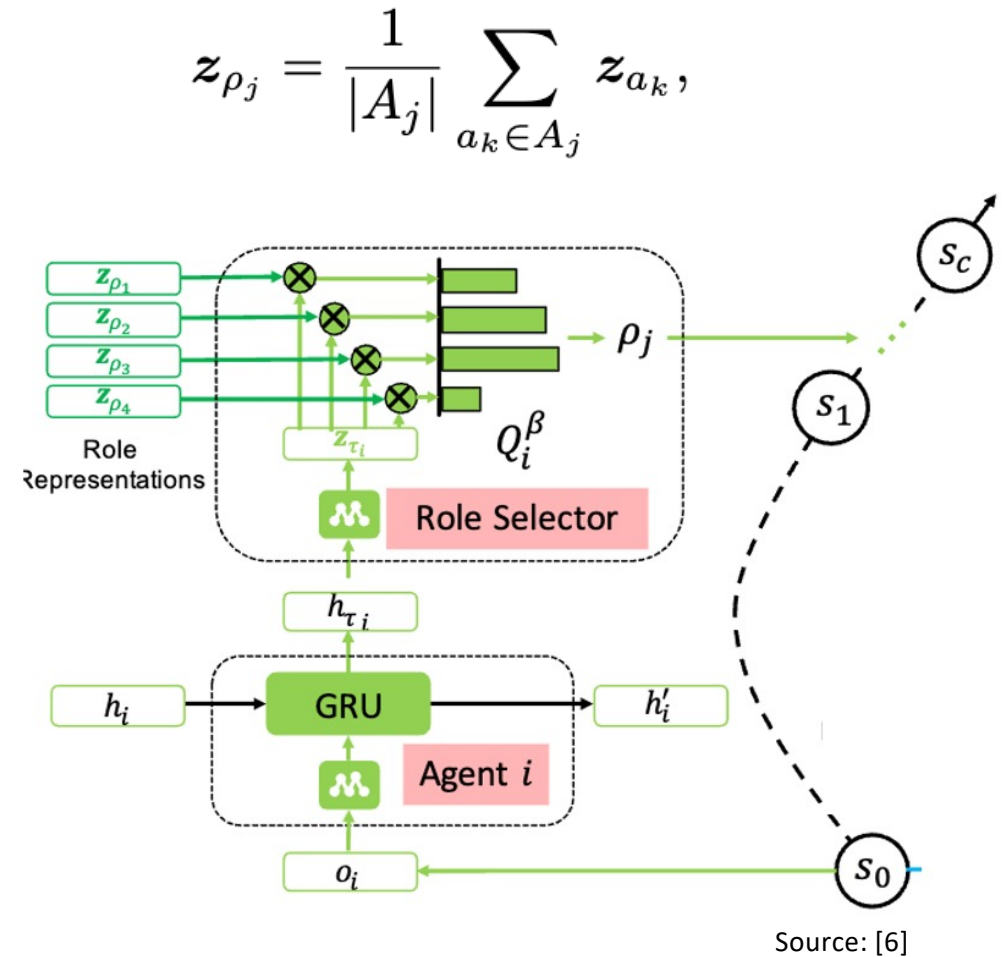
- Agent history τ_i
- Role representations z_{ρ_j}

Process

- $\tau_i \rightarrow \text{LN/GRU} \rightarrow h_{\tau_i} \rightarrow \text{LN} \rightarrow z_{\tau_i}$
- Compare z_{τ_i} with each role representation

Output

- Role value $Q_i^\beta(\tau_i, \rho_j)$
- Select role ρ_j

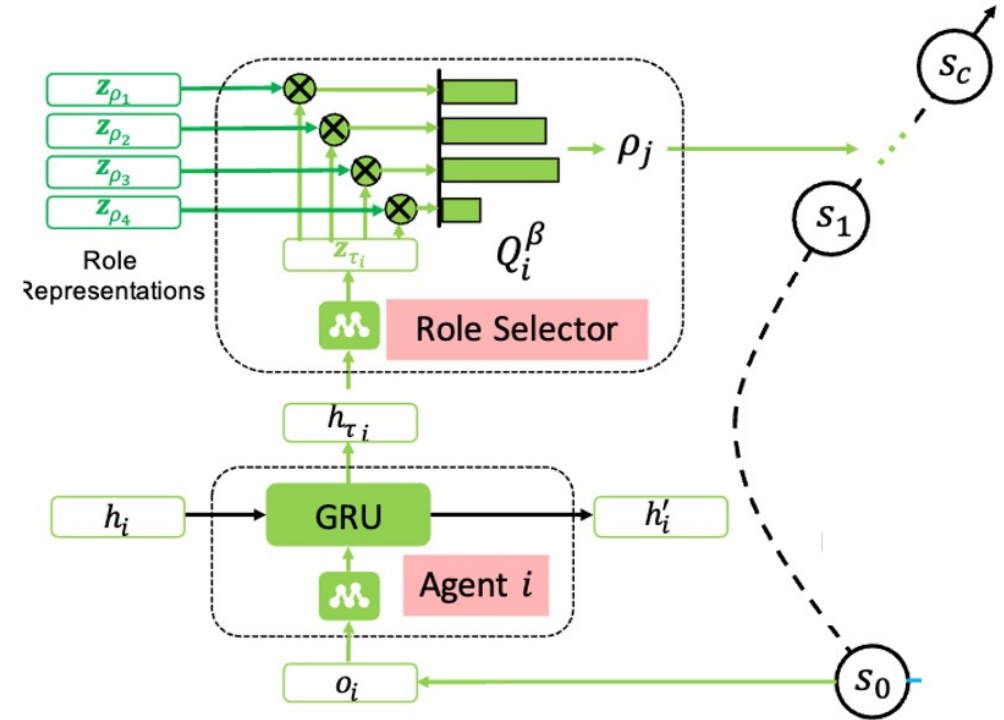


$$Q_i^\beta(\tau_i, \rho_j) = z_{\tau_i}^T z_{\rho_j}$$

B2: Training The Role Selector

Training Objective

- Predict value of current joint role ρ_t
- Compare with reward + future joint role value
- Minimize TD error over c timesteps
- Backprop updates the role selector



Source: [6]

$$\mathcal{L}_\beta(\theta_{\tau_\beta}, \theta_\beta, \xi_\beta) = \mathbb{E}_{\mathcal{D}} \left[\left(\sum_{t'=0}^{c-1} r_{t+t'} + \gamma \max_{\rho'} \bar{Q}_{tot}^\beta(s_{t+c}, \rho') - Q_{tot}^\beta(s_t, \rho_t) \right)^2 \right]$$

Block 3

Role Policy

B3: Role Policy

Goal

Choose the best primitive action inside the selected role

Inputs

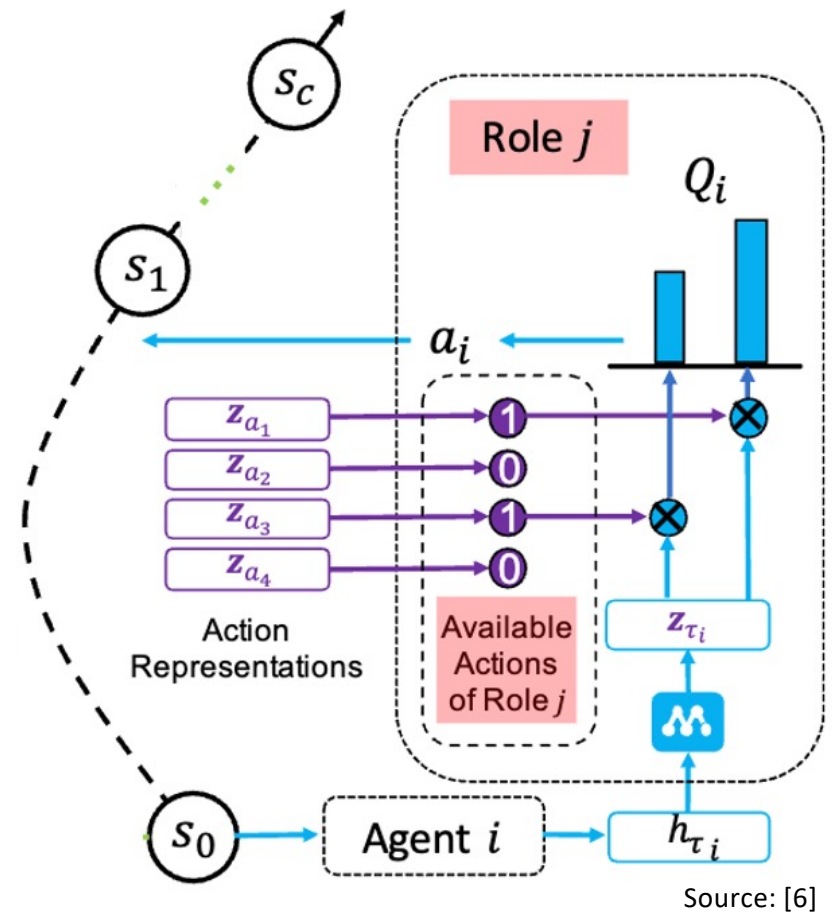
- Selected role ρ_j
- Action representations z_{a_k}

Process

- $\tau_i \rightarrow \text{LN/GRU} \rightarrow h_{\tau_i} \rightarrow \text{LN} \rightarrow z_{\tau_i}$
- Compare z_{τ_i} with action representations

Output

- Action value $Q_i(\tau_i, a_k)$
- Select primitive action a_k

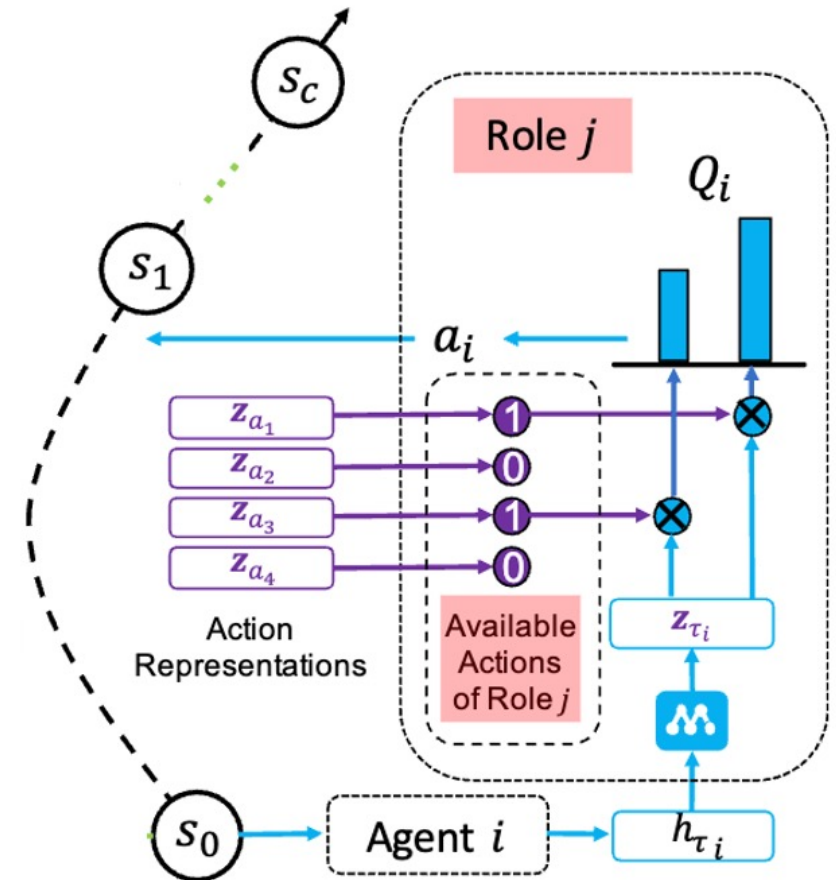


$$Q_i(\tau_i, a_k) = z_{\tau_i}^T z_{a_k}$$

B3: Training The Role Policy

Training Objective

- Predict value of chosen joint action a
- Compare with one-step TD target
- Learn which actions increase team return
- Backprop updates the role policies



Source: [6]

$$\mathcal{L}_\rho(\theta_{\tau_\rho}, \theta_\rho, \xi_\rho) = \mathbb{E}_{\mathcal{D}} \left[\left(r + \gamma \max_{a'} \bar{Q}_{tot}(s', a') - Q_{tot}(s, a) \right)^2 \right]$$

QMIX-style Mixing

Decentralized Agents

Each agent estimates its own utility based strictly on its local history τ_i

$$Q_1, Q_2, \dots, Q_n$$

QMIX Mixing Network

Centralized Training

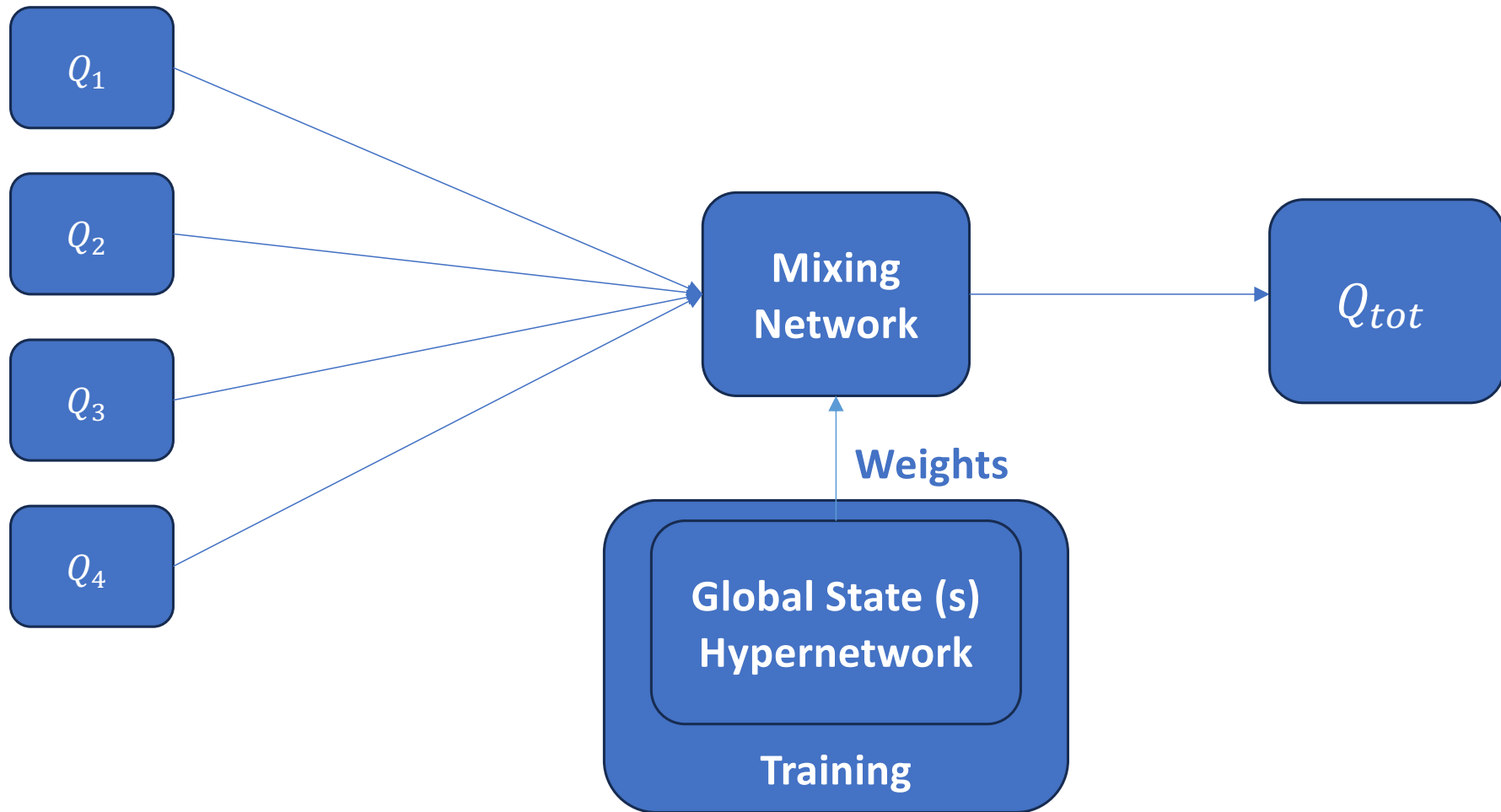
The environment provides a shared team reward, so training needs a global team value Q_{tot}

The RODE Implementation

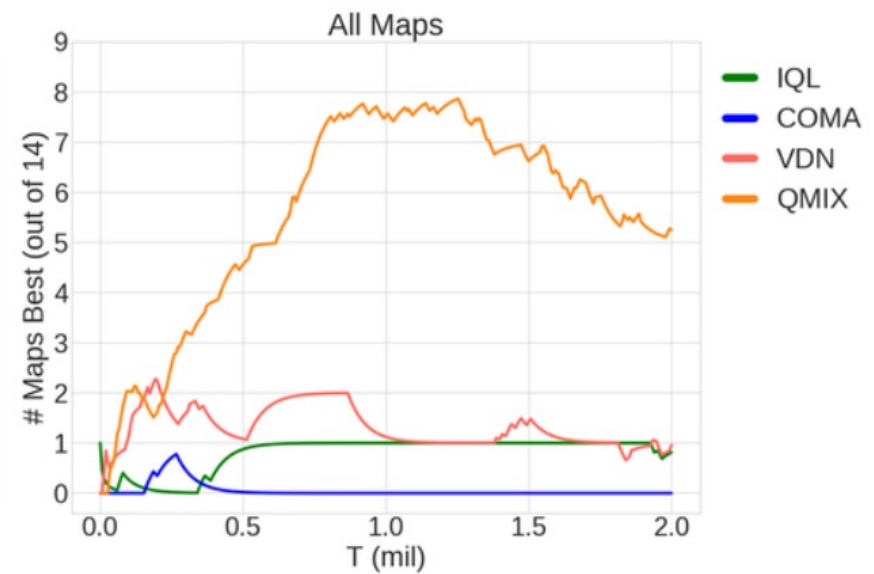
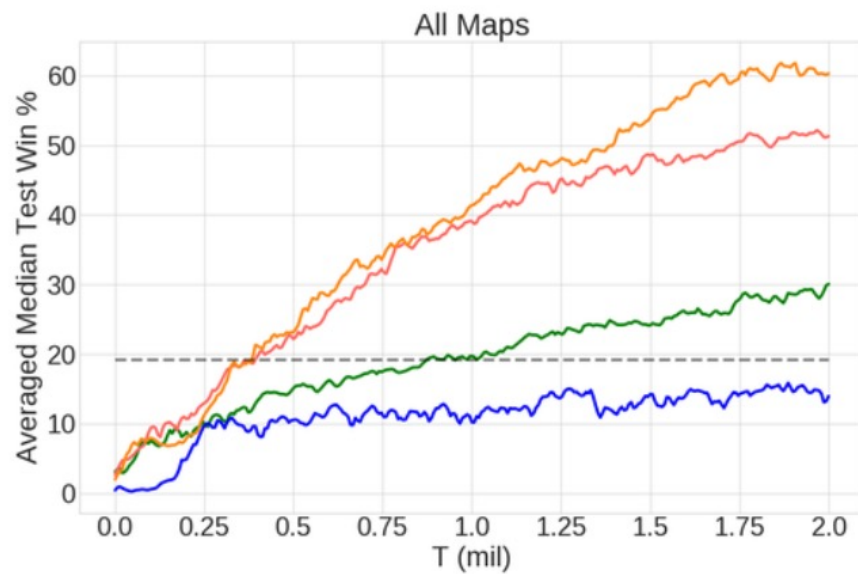
1. **Role Selector Mixing:** Aggregates local role values Q_i^β into a global $Q_{tot}^\beta(s, \rho)$.
2. **Role Policy Mixing:** Aggregates local action values Q_i into a global $Q_{tot}(s, a)$.

QMIX-style Architecture

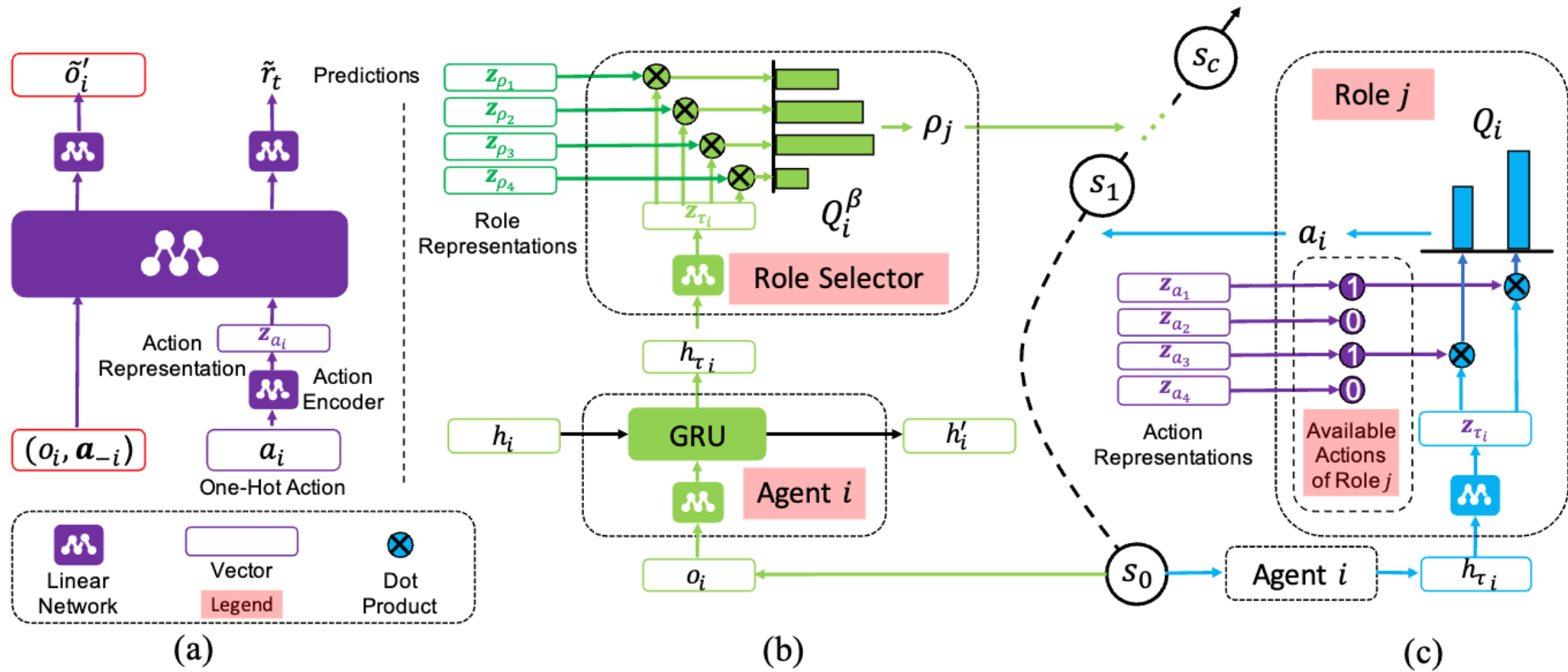
+ Monotonicity Constraint: $\frac{\partial Q_{tot}}{\partial Q_i} \geq 0, \forall i$



Why QMIX Matters



RODE Shortcut



Source: [6]

Without RODE:
Agent chooses from all actions every timestep

With RODE:
Choose role every c steps
→ choose primitive action only inside that role

Implementation Details

Architecture

- shared FC + GRU history encoder
- action representation size $d = 20$
- separate QMIX-style mixers for roles and actions

Training

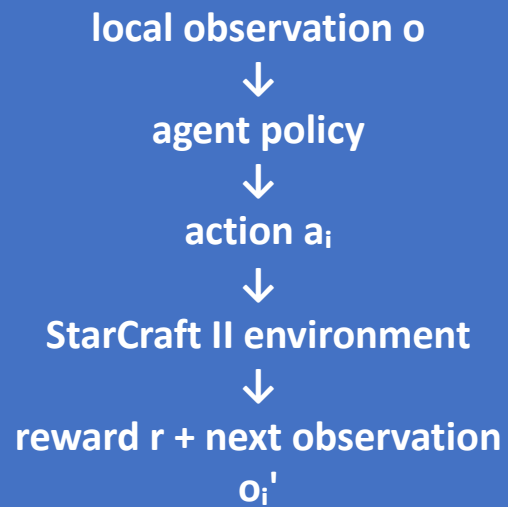
- RMSprop, learning rate 5×10^{-4}
 - batches of 32 full episodes
 - ϵ -greedy exploration

Role Action Spaces

- k-means clustering
- $k = 3$ homogeneous maps
- $k = 5$ heterogeneous maps

+ **Timestep Duration:** Relies on SMAC defaults (8 engine frames per action at 22.4 frames/sec), making 1 MARL timestep equal to ≈ 0.36 seconds of in-game time.^[14]

Validation: StarCraft Multi-Agent Challenge (SMAC)



Source: [1]

Validation: SMAC

Table 1: SMAC challenges.

Name	Ally Units	Enemy Units
2s3z	2 Stalkers & 3 Zealots	2 Stalkers & 3 Zealots
3s5z	3 Stalkers & 5 Zealots	3 Stalkers & 5 Zealots
1c3s5z	1 Colossus, 3 Stalkers & 5 Zealots	1 Colossus, 3 Stalkers & 5 Zealots
5m_vs_6m	5 Marines	6 Marines
10m_vs_11m	10 Marines	11 Marines
27m_vs_30m	27 Marines	30 Marines
3s5z_vs_3s6z	3 Stalkers & 5 Zealots	3 Stalkers & 6 Zealots
MMM2	1 Medivac, 2 Marauders & 7 Marines	1 Medivac, 3 Marauders & 8 Marines
2s_vs_1sc	2 Stalkers	1 Spine Crawler
3s_vs_5z	3 Stalkers	5 Zealots
6h_vs_8z	6 Hydralisks	8 Zealots
bane_vs_bane	20 Zerglings & 4 Banelings	20 Zerglings & 4 Banelings
2c_vs_64zg	2 Colossi	64 Zerglings
corridor	6 Zealots	24 Zerglings

Source: [9]

| Validation: SMAC

Zealot: melee unit

Stalker: ranged unit

Hydralisk: ranged unit

Colossus: strong area-damage unit

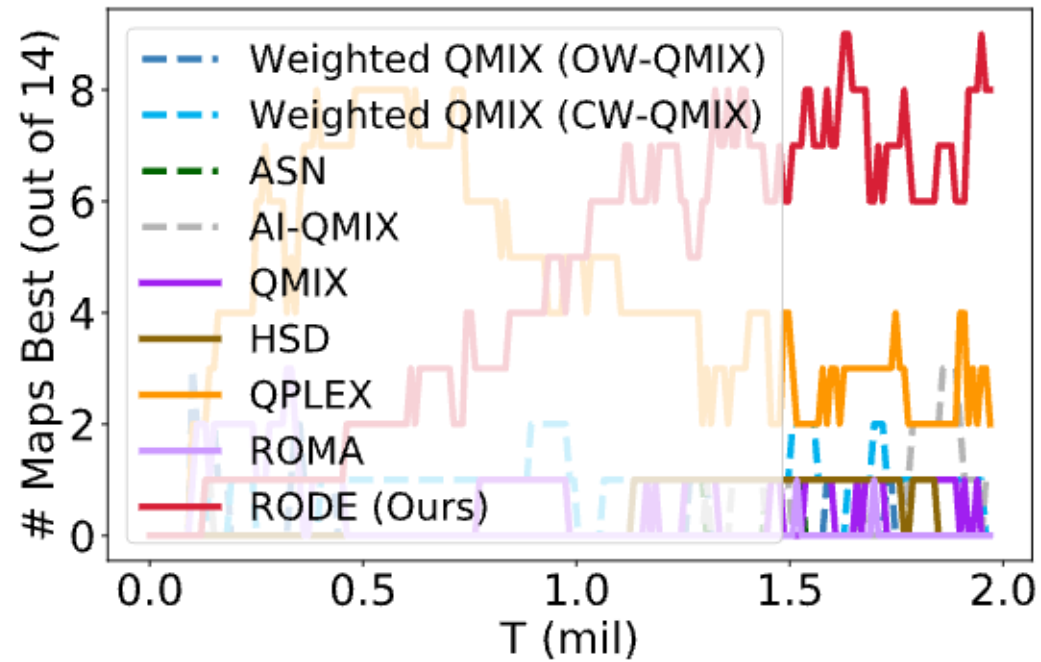
Zergling: weak, fast melee swarm unit

Medivac: healing/support unit

Marauder / Marine: ranged units

+ SMAC Definition: 1 Agent = 1 Individual Unit

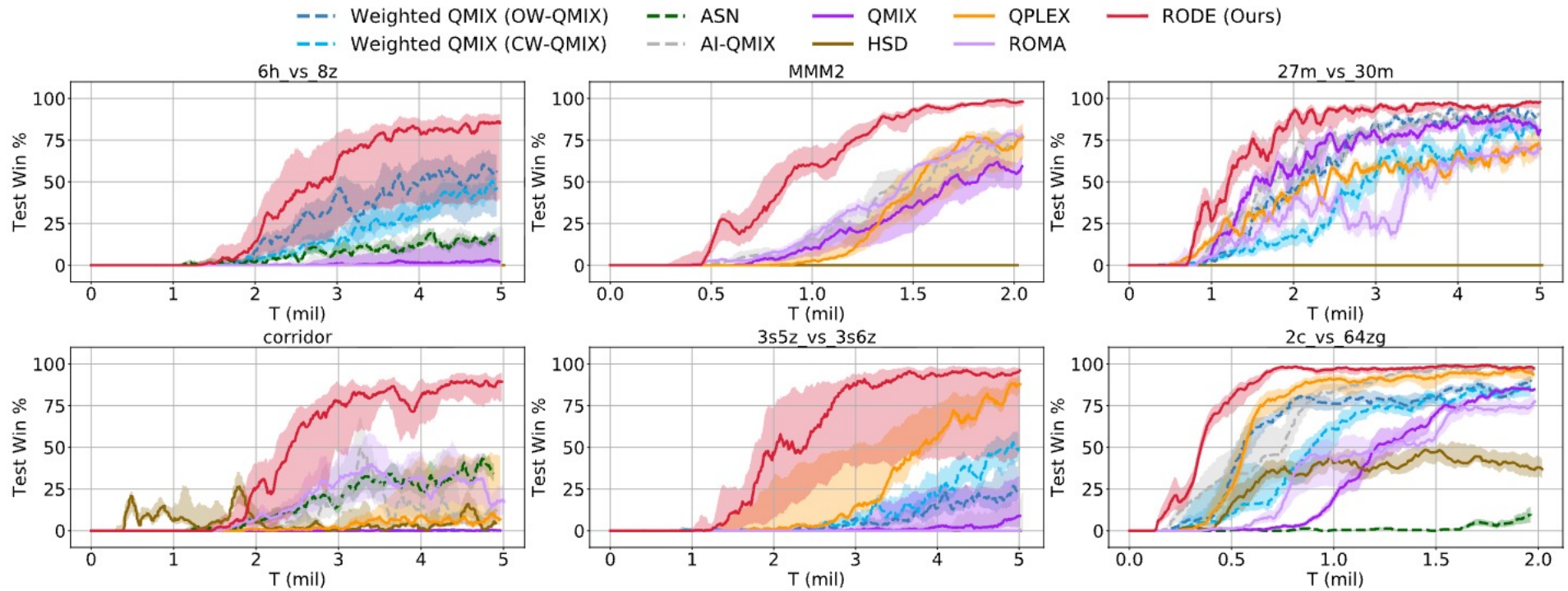
Results



Source: [10]

14 SMAC scenarios

Results



Source: [11]

5 Super Hard Maps / 1 Hard Map (2c_vs_64zg)

| Ablations

RODE components tested:

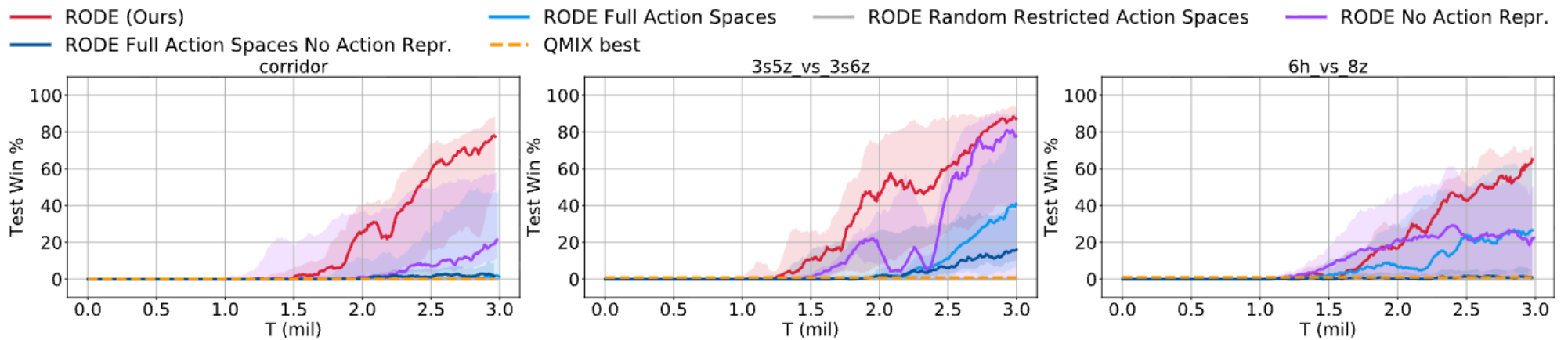
A: restricted role action spaces

B: action-effect clustering

C: action representations in policies / selector

D: hierarchical role structure

Ablations



Source: [12]

restricted role action spaces are the main driver.

Zero-Shot Transfer

- + **Trained on:** Standard corridor map (6 ally Zealots vs 24 enemy Zerglings)
- + **Evaluated on :** Larger, unseen maps scaling up to 18 allies and 36 enemies

# Enemies	24	26	28	30	32	34	36
# Allies							
6	99.2	85.0	25.8	0.0	0.8	0.0	0.0
8	75.0	68.3	6.7	0.0	0.0	0.0	0.0
10	92.5	75.0	24.2	8.3	2.5	1.7	0.0
12	97.5	75.0	74.2	29.2	17.5	9.2	1.7
14	97.5	60.0	69.2	42.5	38.3	22.5	5.8
16	100	55.0	63.3	42.5	48.3	38.3	32.5
18	100	56.7	71.7	54.2	45.8	45.0	50.0

Source: [13]

| Limitations

- + **Static Roles:**

Clusters freeze at $t = 50k$. Any new state-action dynamics discovered later in training cannot reshape role definitions.

- + **Hyperparameter Sensitivity (c):**

If c is too small ($c = 1$), the hierarchy collapses into erratic switching. If c is too large, agents cannot react to dynamic environment changes.

- + **Action Types:** The architecture is restricted to environments with discrete action spaces.

| After RODE

Paper	Limitation	Fix
GoMARL	fixed role structures	dynamic agent grouping
SIRD / SR-MARL	clustering depends on k	structural role discovery
MAZero	sample-inefficient model-free learning	learned model + MCTS planning

| Conclusion

RODE's contribution

- learns action effects before learning roles
- clusters actions into restricted role action spaces
- uses a bi-level hierarchy:
role selector + role policy
- reduces exploration spatially and temporally
- improves sample efficiency on hard SMAC maps

Discussion



Tonghan Wang

to me ▾



Hi Inoussa,

Thank you very much for your kind message and for your interest in our paper. I am glad to hear that you are presenting RODE in your seminar, and I am happy to share a brief perspective.

The main intuition behind RODE was that in many cooperative multi-agent tasks, agents do not need to reason over the full joint action space all the time. Instead, agents often specialize into roles, and each role typically only needs access to a relevant subset of actions. By learning roles automatically, RODE tries to decompose the multi-agent task into simpler sub-problems: agents with similar responsibilities are grouped into roles, and each role learns over a reduced action space. This can make learning more efficient, improve coordination, and encourage meaningful specialization.

Looking forward, I think one promising direction in MARL is to study how such role-based or decomposition-based ideas can scale to more open-ended and generative settings. For example, in generative advertising, one could imagine multiple agents corresponding to different functional roles in the creative pipeline: one agent proposes audience segments, another generates candidate messages, another evaluates brand consistency, another optimizes for engagement or conversion, and another checks constraints such as safety, policy, or fairness. The MARL problem would then be not only about selecting actions in an environment, but about coordinating specialized generative and evaluative agents toward a shared campaign objective.

In that setting and many others, the intuition from RODE may still be useful: not every agent needs to consider every possible action or decision. A copywriting agent does not need the same action space as a budget allocation agent, and a safety-checking agent should operate under a different role than a creative-generation agent. Learning or designing appropriate roles could help make such systems more modular, interpretable, and scalable.

I hope this is helpful for your presentation. Best of luck with the seminar on Friday.

Best regards,

Tonghan

Sources

- [1] StarCraft corridor image, <https://sites.google.com/view/rode-marl>
- [2] Drone image, <https://www.zvut.cz/en/topic/topic-f38144/multi-agent-system-of-drones-and-robots-controlled-by-artificial-intelligence-will-assist-the-army-d282938>
- [3] Reinforcement Learning diagram, Sutton & Barto, Reinforcement Learning: An Introduction
- [4] Tabular Q-Learning and Deep Q-Learning, <https://araffin.github.io/post/rl102/>
- [5] Artificial Intelligence 2 Lecture, Prof. Eduard Kromer, University of Applied Sciences Landshut
- [6] Figure 1, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [7] Figure 2, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [8] Figure 3, The StarCraft Multi-Agent Challenge <https://arxiv.org/pdf/1902.04043>
- [9] Table 1, SMAC Challenges <https://arxiv.org/pdf/1902.04043>
- [10] Figure 4, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [11] Figure 3, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [12] Figure 6, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [13] Figure 5, RODE: Learning Roles to Decompose Multi-Agent Tasks, <https://openreview.net/pdf?id=TTUVg6vkNjK#page=6.92>
- [14] Samvelyan et al. (2019) and Vinyals et al. (2017)

+: Denotes an addition or change made after the presentation.